Q21. Insertion Sort with duplicate elements

Aim: To develop a Python program to insertion sort with duplicate elements and obtain the required output.

Algorithm:

1. Read the array elements.
2. Select the next element or pair of elements to process.
3. Compare values and rearrange them according to the required order.
4. Repeat until the array is ordered, then display the result.

Input:

3 1 4 1 5 9 2 6 5 3

Output:

Sorted: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

```
a = list(map(int, input("Enter array: ").split()))
for i in range(1, len(a)):
    key=a[i]; j=i-1
    while j>=0 and a[j]>key:
        a[j+1]=a[j]; j-=1
    a[j+1]=key
print("Sorted:", a)

```

```
Output

Enter array: 3 1 4 1 5 9 2 6 5 3
Sorted: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]
```

Time Complexity: O(n²)
Space Complexity: O(1)

Result: Therefore, the sorting procedure was carried out and the elements were displayed in the required order.

Q22. Kth positive missing number

Aim: To develop a Python program to kth positive missing number and obtain the required output.

Algorithm:

1. Read the required input values.
2. Use binary search or the required logarithmic search rule to reduce the search range.
3. Store/update the required intermediate values.
4. Display the final answer.

Input:

2 3 4 7 11

5

Output:

Kth missing: 9

```
a = list(map(int, input("Enter sorted array: ").split()))
k = int(input("Enter k: "))
x = 1; count = 0; s = set(a)
while True:
    if x not in s: count += 1
    if count == k: print("Kth missing:", x); break
    x += 1
```

```
Output

Enter sorted array: 2 3 4 7 11
Enter k: 5
Kth missing: 9
```

Time Complexity: O(n)
Space Complexity: O(n)

Result: Therefore, the search procedure was executed and the required element or position was obtained.

Q23. Peak element using O(log n)

Aim: To develop a Python program to peak element using o(log n) and obtain the required output.

Algorithm:

1. Read the sorted input and the target or required position.
2. Set the initial search boundaries.
3. Inspect the middle element and discard the half that cannot contain the answer.
4. Continue until the target is found or the required position is determined; display the answer.

Input:

1 2 3 1

Output:

Peak index: 2

```
a = list(map(int, input("Enter array: ").split()))
lo, hi = 0, len(a)-1
while lo < hi:
    mid=(lo+hi)//2
    if a[mid] < a[mid+1]: lo=mid+1
    else: hi=mid
print("Peak index:", lo)
```

```
Output

Enter array: 1 2 3 1
Peak index: 2
```

Time Complexity: O(log n)
Space Complexity: O(1)

Result: Therefore, the search procedure was executed and the required element or position was obtained.

Q24. First occurrence of a substring

Aim: To develop a Python program to first occurrence of a substring and obtain the required output.

Algorithm:

1. Read the string or list of strings.
2. Examine characters or words in the given order.
3. Apply the required string condition and record matching items.
4. Stop or continue as required, then print the resulting string or value.

Input:

sadbutsad
sad

Output:

Index: 0

```
h = input("Haystack: "); n = input("Needle: ")
print("Index:", h.find(n))
```

```
Output

Haystack: sadbutsad
Needle: sad
Index: 0
```

Time Complexity: O(nm)
Space Complexity: O(1)

Result: Therefore, the string-processing condition was checked and the required output was produced.

Q25. Strings that are substrings of another word

Aim: To develop a Python program to strings that are substrings of another word and obtain the required output.

Algorithm:

1. Read the string or list of strings.
2. Examine characters or words in the given order.
3. Apply the required string condition and record matching items.
4. Stop or continue as required, then print the resulting string or value.

Input:

mass as hero superhero

Output:

['as', 'hero']

```
words = input("Enter words: ").split()
ans = []
for i,w in enumerate(words):
    if any(i!=j and w in x for j,x in enumerate(words)):
        ans.append(w)
print(ans)
```

```
Output

Enter words: mass as hero superhero
['as', 'hero']
```

Time Complexity: O(n²L)
Space Complexity: O(n)

Result: Therefore, the string-processing condition was checked and the required output was produced.

Q26. Closest pair of points – brute force

Aim: To develop a Python program to closest pair of points – brute force and obtain the required output.

Algorithm:

1. Read the input array and any additional parameters.
2. Initialize counters, sets, or variables needed for the calculation.
3. Traverse the relevant elements and apply the condition stated in the problem.
4. Print the final count, sum, selected elements, or other required value.

Input:

4
1 2
4 5
7 8
3 1

Output:

Closest pair and distance

```
import math
pts = [tuple(map(int, input("Enter x y: ").split())) for _ in
range(int(input("Number of points: ")))]
best = math.inf; pair=None
for i in range(len(pts)):
    for j in range(i+1,len(pts)):
      d=math.dist(pts[i],pts[j])
      if d<best: best=d; pair=(pts[i],pts[j])
print("Closest pair:", pair); print("Distance:", best)
```

```
Output

Number of points: 4
Enter x y: 1 2
Enter x y: 4 5
Enter x y: 7 8
Enter x y: 3 1
Closest pair: ((1, 2), (3, 1))
Distance: 2.23606797749979
```

Time Complexity: O(n²)
Space Complexity: O(1)

Result: Therefore, the array was processed according to the given condition and the required result was displayed.

Q27. Convex Hull – brute force

Aim: To develop a Python program to convex hull – brute force and obtain the required output.

Algorithm:

1. Read the input data required by the problem.
2. Initialize variables or data structures needed for processing.
3. Process the input using the rule specified in the problem statement.
4. Display the computed result.

Input:

5
1 1
4 6
8 1
0 0
3 3

Output:

Convex hull

```
def cross(a,b,c):
    return (b[0] a[0])*(c[1] a[1]) (b[1] a[1])*(c[0] a[0])

pts [tuple(map(int,input("x y: ").split())) for _ in range(int(input("Number of points: "))))]
h=[]
for p in sorted(pts):
    while len(h)> 2 and cross(h[-2],h[-1],p)< 0: h.pop()
    h.append(p)
l=h
h=[]
for p in sorted(pts,reverse True):
    while len(h)> 2 and cross(h[-2],h[-1],p)< 0: h.pop()
    h.append(p)
print("Convex hull:",l[: 1]+h[: 1])
```

```
Output

Number of points: 5
x y: 1 1
x y: 4 6
x y: 8 1
x y: 0 0
x y: 3 3
Convex hull: [(0, 0), (8, 1), (4, 6)]
```

Time Complexity: O(n²)
Space Complexity: O(n)
Result: Therefore, the problem was processed using the stated steps and the output was obtained successfully.

Q28. Traveling Salesperson Problem – exhaustive search

Aim: To develop a Python program to traveling salesperson problem – exhaustive search and obtain the required output.
Algorithm:

1. Read the sorted input and the target or required position.
2. Set the initial search boundaries.
3. Inspect the middle element and discard the half that cannot contain the answer.
4. Continue until the target is found or the required position is determined; display the answer.

Input:

4
1 2
4 5
7 1
3 6
Output:

Minimum distance and path

```
import itertools, math
pts=[tuple(map(int,input("x y: ").split())) for _ in range(int(input("Cities: ")))]
best=math.inf
for p in itertools.permutations(range(1,len(pts))):
    r=(0,)+p+(0,)
    d=sum(math.dist(pts[r[i]],pts[r[i+1]]) for i in range(len(r)-1))
    if d<best: best,path=d,r
print("Minimum distance:",best)
print("Path:",[pts[i] for i in path])
```

```
Output

Cities: 4
x y: 1 2
x y: 4 5
x y: 7 1
x y: 3 6
Minimum distance: 16.969112047670894
Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]
```

Time Complexity: O(n!)
Space Complexity: O(n)
Result: Therefore, the search procedure was executed and the required element or position was obtained.

Q29. Assignment Problem – exhaustive search

Aim: To develop a Python program to assignment problem – exhaustive search and obtain the required output.

Algorithm:

1. Read the sorted input and the target or required position.
2. Set the initial search boundaries.
3. Inspect the middle element and discard the half that cannot contain the answer.
4. Continue until the target is found or the required position is determined; display the answer.

Input:

3
3 10 7
8 5 12
4 6 9

Output:

Assignment and minimum cost: 19

```
import itertools
cost = []
n = int(input("Number of workers: "))
for _ in range(n): cost.append(list(map(int,input("Cost row: ").split())))
best=10**18; ans=None
for p in itertools.permutations(range(n)):
    c = sum(cost[i][p[i]] for i in range(n))
    if c<best: best=c; ans=p
print("Assignment:", ans); print("Minimum cost:", best)
```

```
Output

Number of workers: 3
Cost row: 3 10 7
Cost row: 8 5 12
Cost row: 4 6 9
Assignment: (2, 1, 0)
Minimum cost: 16
```

Time Complexity: O(n!)
Space Complexity: O(n)

Result: Therefore, the search procedure was executed and the required element or position was obtained.

Q30. 0-1 Knapsack – exhaustive search

Aim: To develop a Python program to 0-1 knapsack – exhaustive search and obtain the required output.

Algorithm:

1. Read the sorted input and the target or required position.
2. Set the initial search boundaries.
3. Inspect the middle element and discard the half that cannot contain the answer.
4. Continue until the target is found or the required position is determined; display the answer.

Input:

2 3 1
4 5 3
4

Output:

Selected: [0, 2], Value: 7

Time Complexity: O(2^n)
Space Complexity: O(n)

```
import itertools
w=list(map(int,input("Weights: ").split()))
v=list(map(int,input("Values: ").split()))
cap=int(input("Capacity: ")); best=0; chosen=[]
for mask in range(1<<len(w)):
    ids=[i for i in range(len(w)) if mask>>i&1]
    if sum(w[i] for i in ids)<=cap and sum(v[i] for i in ids)>best:
        best=sum(v[i] for i in ids); chosen=ids
print("Selected:", chosen); print("Value:", best)
```

Output

```
Weights: 2 3 1
Values: 4 5 3
Capacity: 4
Selected: [1, 2]
Value: 8
```

Result: Therefore, the search procedure was executed and the required element or position was obtained.